

Simon Gao

Email: example@gmail.com | Phone: 5042611911

Summary

- Over **10 years of experience** as a Full Stack Java Developer designing, developing, and supporting scalable enterprise web applications using modern Java technologies.
- Extensive experience across the complete **Software Development Life Cycle (SDLC)** using **Agile (Scrum)** and **Waterfall** methodologies, following software engineering best practices including **Test Driven Development (TDD)** and code reviews.
- Strong experience developing **RESTful APIs** and Microservices using **Java 17, Spring Boot 3, Spring MVC, Spring Data JPA, Spring IoC, Spring AOP, Spring Security, Hibernate, and Maven**.
- Hands-on experience implementing secure applications using **Spring Security, JWT, OAuth 2.0**, role-based authorization, authentication, and secure REST API development.
- Extensive experience building **Single Page Applications (SPA)** using **Angular 17, TypeScript, RxJS, Signals, Reactive Forms, Routing, Dependency Injection, Services, Pipes, Directives, Guards, Interceptors, and HttpClient**.
- Strong frontend development experience using **HTML5, CSS3, Bootstrap, JavaScript, TypeScript**, and responsive UI design principles.
- Strong knowledge of **Core Java** and **Object-Oriented Programming (OOP)** concepts including Collections Framework, Multithreading, Concurrency, Exception Handling, File I/O, JVM Memory Management, and Garbage Collection.
- Hands-on experience using modern Java features including **Lambda Expressions, Functional Interfaces, Stream API, Optional, Generics, Records, CompletableFuture, Virtual Threads**, and the **Java Time API**.
- Experience implementing enterprise design patterns including **DAO, DTO, Factory, Singleton, Strategy, Service Locator, Session Facade, Builder, Observer, and Dependency Injection** patterns.
- Strong experience working with **SQL** and **NoSQL** databases including **MySQL, PostgreSQL, MongoDB, Cassandra, and Redis**, using **JDBC, Hibernate ORM, Spring Data JPA, and Spring Data MongoDB**.
- Experience designing normalized relational databases, writing complex SQL queries, stored procedures, triggers, indexing strategies, query optimization, and database performance tuning.
- Hands-on experience developing **Microservices** using **Spring Cloud, Eureka Service Discovery, API Gateway, Ribbon**, centralized configuration with **Spring Cloud Config**, and fault tolerance using **Resilience4j Circuit Breaker**.
- Experience implementing asynchronous and event-driven architectures using **Apache Kafka, Amazon SNS, Amazon SQS, and CompletableFuture** for high-performance distributed systems.
- Hands-on experience deploying cloud-native applications on **AWS**, including **EC2, ECS, ECR, S3, RDS, Lambda, API Gateway, IAM, SNS, SQS, CloudWatch, and VPC**.
- Experience containerizing applications using **Docker**, orchestrating deployments with **Kubernetes (K8s)**, and managing cloud deployments through **Amazon ECS**.
- Strong experience building and maintaining **CI/CD pipelines** using **Jenkins, GitHub Actions, Docker**, and automated deployment workflows.

- Experience documenting REST APIs using **OpenAPI / Swagger** and collaborating with frontend and QA teams through API-first development.
- Strong understanding of software testing with **JUnit 5, Mockito, Spring Boot Test, Testcontainers, WireMock**, integration testing, and application logging using **SLF4J** and **Log4j**.
- Proficient in source code management using **Git, GitHub**, branching strategies, pull requests, and collaborative development workflows.
- Experience using **Jira, Confluence**, and Agile ceremonies including sprint planning, backlog refinement, daily stand-ups, sprint reviews, and retrospectives.
- Strong analytical, troubleshooting, and communication skills with the ability to quickly learn new technologies and deliver high-quality software solutions in fast-paced Agile environments.

Technical Skills

Programming Languages: Java 8/11/17/21, J2EE, SQL, JavaScript, TypeScript

Web Technologies: Spring Boot, Spring MVC, Spring Security, Spring Data JPA, Hibernate, RESTful APIs, Microservices, J2EE, Servlets, JSP, HTML5, CSS3, Bootstrap, Angular 17, RxJS, Signals, JSON, XML

Frameworks: Spring Boot, Spring Cloud, Spring MVC, Spring Security, Spring Data JPA, Hibernate ORM, JPA, Angular, JUnit 5, Mockito

Databases: MySQL, PostgreSQL, MongoDB, Cassandra, Redis, DynamoDB, Elasticsearch

Cloud & Microservices: AWS, Spring Cloud, Eureka, API Gateway, Ribbon, OpenFeign, Resilience4j, Kafka, SNS, SQS, Docker, Kubernetes (K8s), ECS, ECR

AWS Services: EC2, ECS, ECR, S3, RDS, DynamoDB, IAM, Lambda, SNS, SQS, API Gateway, CloudWatch, VPC

CI/CD & DevOps: Jenkins, GitHub Actions, Maven, Docker, SonarQube, JaCoCo, Testcontainers, WireMock

Build Tools: Maven, Gradle

Development Tools: IntelliJ IDEA, Eclipse, Visual Studio Code, Postman, Swagger/OpenAPI, Git, GitHub, Jira, Confluence

Operating Systems: Linux, Unix, Windows, macOS

Version Control: Git, GitHub

Testing & Logging: JUnit 5, Mockito, Spring Boot Test, Testcontainers, WireMock, SLF4J, Log4j

Experience

Client Name: Wells Fargo Charlotte, NC/US

Senior Full Stack Developer

Oct 2023 – Present

Project: Credit Card Platform — Risk & Underwriting Subsystem

Description: Built and maintained Wells Fargo's Risk & Underwriting Subsystem — an event-driven microservice cluster covering credit scoring, fraud detection, identity verification, income verification, AML/sanctions screening, and underwriting decisioning.

- Participated in all phases of the SDLC including requirement analysis, design, development, testing, deployment, and production support using Agile/SCRUM methodology.
- Developed and maintained RESTful Web Services using **Java 17, Spring Boot 3, Spring Data JPA, Hibernate**, and **Spring MVC** within a Microservices architecture.

- Built responsive user interfaces using **Angular 17**, **HTML5**, **CSS3**, **Bootstrap**, and **TypeScript** for internal risk review and underwriting management dashboards.
- Applied Core Java concepts including **Collections**, **Multithreading**, **Concurrency**, **Lambda Expressions**, **Stream API**, **Optional**, **Generics**, and **CompletableFuture**.
- Developed six risk microservices within the subsystem: Credit Scoring (**PostgreSQL**), Fraud Detection (**Redis** + **DynamoDB**), Identity Verification (**DocumentDB**), Income Verification (**MongoDB**), AML/Sanctions (**PostgreSQL**), and Underwriting/Decision Service implementing a Drools rule engine for policy-driven approve/decline decisioning.
- Built the Risk Scoring Aggregator to fan-in parallel results from all six risk services using client-request-id correlation, producing a unified underwriting decision delivered to the Account & Card Issuance layer via sync exception path and async **Kafka** events.
- Implemented asynchronous event-driven communication using **Apache Kafka**; risk services subscribed as consumers to application events from the upstream Product & Application Layer and published scored results to downstream aggregation topics.
- Utilized **Redis** within the Fraud Detection Service for real-time velocity checks and fraud pattern caching, reducing repeated **DynamoDB** lookups during high-volume application bursts.
- Developed persistence layers across heterogeneous data stores — **PostgreSQL** for Credit Scoring, AML/Sanctions, and Underwriting; **DynamoDB** for fraud signals; **DocumentDB** for identity records; **MongoDB** for income verification data.
- Integrated **Google Cloud Platform (GCP) BigQuery** and **Vertex AI** for risk model training and scoring analytics, enabling data-driven refinement of credit scoring weights and fraud detection thresholds.
- Enhanced **AML/Sanctions** screening rules to incorporate **OFAC** updates related to Russia-Ukraine conflict sanctions, ensuring real-time compliance with expanding SDN watchlists and correspondent banking restrictions.
- Updated credit scoring thresholds and underwriting policy rules in response to the Federal Reserve's rate hike cycle, adjusting **APR calculation logic** and tightening approval criteria to reflect increased credit risk.
- Implemented **Spring Security** with **JWT** and **OAuth 2.0** to secure internal risk APIs and enforce role-based access control across underwriting and compliance services.
- Built microservices using **Spring Cloud**, **Eureka** Service Discovery, **API Gateway**, **OpenFeign**, and **Resilience4j** Circuit Breaker to improve fault tolerance and service communication within the risk cluster.
- Deployed applications on **Microsoft Azure** using **Azure Kubernetes Service (AKS)**, **Azure Container Registry (ACR)**, **Azure Virtual Machines**, **Azure Blob Storage**, **Azure SQL Database**, and **Azure Active Directory**, supporting secure and highly available cloud infrastructure within a private VNet.
- Containerized all risk microservices using **Docker** and orchestrated deployments through **Azure Kubernetes Service (AKS)**, enabling automated scaling, rolling updates, and self-healing across the risk cluster.
- Developed **CI/CD pipelines** using **Jenkins**, **Maven**, and **Docker** to automate build, testing, and deployment across all risk services.
- Wrote unit and integration tests using **JUnit 5**, **Mockito**, and **Spring Boot Test** to ensure reliability of scoring logic and aggregation outcomes.
- Created **REST API** documentation using **Swagger/OpenAPI** to support collaboration between risk, compliance, and frontend teams.

- Used **Git**, **Bitbucket**, and **JIRA** for source code management, sprint planning, and defect tracking.

Environment: Java 17, Spring Boot 3, Spring MVC, Spring Security, Spring Data JPA, Hibernate, Spring Cloud, Eureka, API Gateway, OpenFeign, Resilience4j, Drools, Angular 17, HTML5, CSS3, Bootstrap, TypeScript, RxJS, Signals, PostgreSQL, Redis, DynamoDB, DocumentDB, MongoDB, Apache Kafka, Microsoft Azure (AKS, ACR, Azure VM, Blob Storage, Azure SQL, Azure AD), GCP (BigQuery, Vertex AI), Docker, Kubernetes, Maven, Jenkins, JUnit 5, Mockito, Git, Bitbucket, JIRA, Swagger/OpenAPI.

Client Name: Costco Wholesale, Issaquah, WA/US

Senior Full Stack Developer

Sep 2019 – Sep 2023

Project: E-Commerce Order & Inventory Management System

Description: Developed and enhanced Costco's e-commerce platform by building scalable microservices for product catalog, inventory management, shopping cart, and order fulfillment. Improved application performance, system reliability, and customer shopping experience during peak shopping seasons.

- Delivered full **SDLC** coverage from requirement analysis to production support using **Agile/SCRUM** methodology.
- Built and deployed scalable **RESTful APIs** using **Java 11** (upgraded to **Java 17**), **Spring Boot 2.5**, **Spring Data JPA**, **Hibernate**, **MySQL**, and **Redis** to support high-volume e-commerce transactions.
- Built responsive web applications using **Angular 13**, **HTML5**, **CSS3**, **Bootstrap**, and **TypeScript** for product browsing, shopping cart, inventory inquiry, and order management.
- Applied **Core Java** concepts including **Collections**, **Multithreading**, **Concurrency**, **Exception Handling**, **File I/O**, and modern Java features such as **Lambda Expressions**, **Functional Interfaces**, **Stream API**, **Optional**, **Generics**, and **CompletableFuture**.
- Developed business components using **Spring IoC**, **Dependency Injection**, **Spring AOP**, and annotation-based configuration to improve maintainability and modularity.
- Secured customer-facing and administrative REST APIs by configuring **Spring Security** with **JWT**-based authentication and role-based access control policies.
- Developed persistence layers using **Hibernate ORM** and **Spring Data JPA** across **MySQL** and **Oracle Database**, optimizing SQL queries and improving database performance for order and inventory processing.
- Developed membership-based order management features supporting Costco's subscription model, including member verification, exclusive pricing, and purchase limit enforcement per membership tier.
- Improved platform response time by implementing **Redis** distributed caching for product catalog and inventory data, reducing database load during high-traffic shopping periods.
- Scaled the e-commerce platform to handle a surge in online orders during the COVID-19 pandemic, implementing caching strategies and load balancing to maintain system availability under peak traffic conditions.
- Integrated **Elasticsearch** to provide high-performance product search with keyword search, filtering, and sorting capabilities.
- Implemented **Apache Kafka** producers and consumers to process order creation, inventory updates, payment confirmation, and shipment notifications through asynchronous event-driven communication.
- Built Microservices using **Spring Cloud**, **Eureka Service Discovery**, **API Gateway**, **OpenFeign**, and **Resilience4j Circuit Breaker** to improve service communication, scalability, and fault tolerance.

- Migrated e-commerce workloads to a multi-cloud environment using Google Cloud Platform (GCP) and Microsoft Azure, leveraging GCP Cloud Storage for product media assets and Azure Virtual Machines for application hosting across peak retail seasons.
- Containerized microservices using **Docker**, orchestrated deployments through **Google Kubernetes Engine (GKE)**, and built **CI/CD pipelines** using **Jenkins** and **Maven** to automate build, testing, and zero-downtime releases during peak shopping seasons.
- Created REST API documentation using **Swagger/OpenAPI** to improve API integration and collaboration across development teams.
- Validated application quality by writing unit and integration tests using **JUnit 5**, **Mockito**, and **Spring Boot Test**, achieving reliable coverage across order processing and inventory management workflows.
- Managed source code versioning and sprint workflows using **Git**, **GitHub**, and **JIRA**, conducting regular code reviews to maintain code quality standards.

Environment: Java 11, Java 17, Spring Boot 2.5.0, Spring MVC, Spring Security, Spring Data JPA, Hibernate, Spring Cloud, Eureka, API Gateway, OpenFeign, Resilience4j, Angular 13, HTML5, CSS3, Bootstrap, TypeScript, RxJS, MySQL, Redis, Elasticsearch, Apache Kafka, GCP (GKE, Cloud Run, Cloud Storage, Cloud SQL), Microsoft Azure (Azure VM, Azure Blob Storage), Kubernetes, Docker, Maven, Jenkins, JUnit 5, Mockito, Git, GitHub, JIRA, Swagger/OpenAPI.

Client Name: Nike China, Shanghai, China

Java Full Stack Developer

Jul 2016 – July 2019

Project: E-Commerce Order & Inventory Management Platform

Description: Developed and maintained Nike China's e-commerce platform by building enterprise web applications for product catalog, inventory management, order processing, and customer account management. Improved application performance, system reliability, and customer shopping experience.

- Designed and implemented enterprise **RESTful Web Services** using **Java 8**, **Spring Boot**, **Spring MVC**, **Spring Data JPA**, **Hibernate**, and **MySQL** to power product catalog, shopping cart, and order processing workflows.
- Developed platform features aligned with Nike's Direct-to-Consumer (DTC) strategy, supporting Nike.com and Nike App order fulfillment in the China market.
- Built responsive user interfaces using **Angular 2**, **Angular 4**, and **Angular 6** later, **HTML5**, **CSS3**, **Bootstrap**, and **TypeScript**, implementing Routing, Reactive Forms, Dependency Injection, RxJS, and HttpClient.
- Optimized the platform for mobile-first shopping behavior in the China market, supporting high mobile traffic ratios across product browsing and checkout flows.
- Applied Core Java concepts including Collections, Multithreading, Exception Handling, Lambda Expressions, Stream API, Optional, and Generics, while developing business components with Spring IoC, Dependency Injection, and Spring AOP.
- Applied **Spring Security** with **JWT** authentication to protect e-commerce endpoints, and built persistence layers using **Hibernate ORM** and **Spring Data JPA** across **MySQL** and **Oracle Database**, optimizing SQL queries to improve application performance.
- Integrated **Redis** as an in-memory cache to accelerate product and inventory data retrieval, and implemented **Apache Kafka** for asynchronous order processing, inventory synchronization, and notification services.
- Supported high-traffic e-commerce events including Singles' Day (11.11) promotions, handling peak order volumes and inventory spikes across Nike China's digital platform.
- Developed microservices using **Spring Cloud** and **Eureka Service Discovery**, improving application scalability and service communication.

- Automated the application build and deployment lifecycle using **Jenkins**, **Maven**, and **Docker**, establishing **CI/CD** pipelines to accelerate release cycles and reduce manual deployment errors.
- Deployed applications on **AWS** using **EC2**, **S3**, and **RDS**, supporting scalable and reliable cloud infrastructure for Nike China's e-commerce platform.
- Created REST API documentation using **Swagger** and developed unit and integration tests using **JUnit** and **Mockito** to ensure application quality and reliability.
- Collaborated across development teams using **Git**, **GitLab**, and **JIRA** for version control, issue tracking, and Agile sprint coordination.

Environment: Java 8, Spring Boot, Spring MVC, Spring Security, Spring Data JPA, Hibernate, Spring Cloud, Eureka, Angular 2, Angular 4 and Angular 6, HTML5, CSS3, Bootstrap, TypeScript, RxJS, MySQL, Redis, Apache Kafka, Docker, Maven, Jenkins, JUnit, Mockito, Git, GitLab, JIRA, Swagger, AWS EC2, S3, RDS

Education

Tulane University	PhD in Economics
University of Wisconsin-Madison	Master of Science in Quantitative Economics
Capital Normal University	Bachelor of Science in Statistics